

Module 03: Control Flow

Loops and conditions in Java hold few surprises for anyone who has written code before. Three things here are worth your attention: Java has no truthiness, the modern `switch` is genuinely better than the one you may have met, and labelled break is a piece of syntax most languages do not have.

What You'll Learn

- Why `if (name)` does not compile, and why that is a feature
- Short-circuit evaluation as a correctness tool, not an optimisation
- Switch expressions, multi-label cases, and `yield`
- Exhaustiveness checking, and why you often want no `default` branch
- Every loop form, and which one to reach for
- Labelled break, for the nested-loop case

Coming From Another Language

	Python / JS	Java
Condition	any value	<code>boolean</code> only
Empty string is falsy	yes	there is no falsy
<code>switch</code> on strings	JS yes, Python 3.10+ match	yes
Fall-through	JS yes	only in the old statement form
Break out of nested loops	flag variable	<code>break label;</code>
<code>for x in xs</code>	native	<code>for (var x : xs)</code>

The single biggest adjustment is the first row. Java will not let you write `if (items)` and decide for you what an empty list means.

The Lesson

No truthiness

```
String name = "";
if (name) { }    // does not compile
if (0) { }       // does not compile
```

You write the comparison out:

```
if (!name.isEmpty()) { ... }
```

More typing, and in exchange there is no table of falsy values to memorise and no debate about whether `"0"` or `[]` counts as false. Every conditional says exactly what it tests.

Short-circuit as a guard

`&&` skips its right operand when the left one already decided the answer. That is not only a performance detail, it is how you guard against null:

```
if (maybe != null && maybe.length() > 3) { ... }
```

Reverse the two and it throws. The order of `&&` operands is a correctness decision.

`&` and `|` also work on booleans and do **not** short-circuit. You will almost never want them there. Their real job is bitwise work:

```
6 & 3 = 2      6 | 3 = 7      6 ^ 3 = 5      6 << 1 = 12
```

Switch, the good version

A switch **expression** produces a value. See [SwitchExpressions.java](#):

```
String kind = switch (day) {  
    case 1, 2, 3, 4, 5 -> "weekday";  
    case 6, 7 -> "weekend";  
    default -> "not a day";  
};
```

No `break`, several labels per branch, and the whole thing is an expression you can assign. When a branch needs more than one statement, use a block and `yield`:

```
int workload = switch (day) {  
    case 6, 7 -> {  
        int base = 10;  
        yield base * 2;  
    }  
    default -> 8;  
};
```

`yield` exists because `return` would try to leave the entire method.

Exhaustiveness, the part that actually matters

This method has no `default`, and the compiler accepts it:

```
String describe(Light light) {
    return switch (light) {
        case RED -> "stop";
        case AMBER -> "get ready";
        case GREEN -> "go";
    };
}
```

Three constants, three branches, complete. Now delete the `GREEN` branch, as `SwitchMustBeExhaustive.java` does:

```
error: the switch expression does not cover all possible input values
    return switch (light) {
        ^
missing patterns:
    Light.GREEN
```

The compiler names the case you forgot.

This leads to advice that sounds backwards at first:

Leave out the `default` branch when the input is a closed set.

With no `default`, adding a fourth constant to `Light` breaks compilation at every switch that needs updating, and the error points at each one. With a `default`, all of them keep compiling and quietly route the new constant to the fallback. A compile error you get today beats a bug you find in production.

Fall-through, and why it went away

The old statement form runs into the following cases until something stops it.

`WhyFallThroughWentAway.java` shows both, with `n = 2`:

```
n = 2, old statement form:
    two
    three
n = 2, arrow form:
    two
```

The old form matched case 2, printed, and then fell into case 3 because there was no `break`. One missing keyword, and the wrong behaviour reads as correct.

Fall-through was occasionally used to group cases. The arrow form does that with a comma and no risk:

```
case 1, 2, 3 -> handleSmall();
```

Loops

Four forms, and the choice is usually obvious. From `Loops.java`:

Form	Use it when
<code>for (var x : xs)</code>	you do not need the index. Most of the time.
<code>for (int i = 0; ...)</code>	you need the index, a step other than 1, or to go backwards
<code>while</code>	the iteration count is not known up front
<code>do-while</code>	the body must run at least once. Rare.

Labelled break is the one piece of loop syntax with no equivalent in most languages. It exits an outer loop from inside an inner one:

```
search:
for (int row = 0; row < grid.length; row++) {
    for (int col = 0; col < grid[row].length; col++) {
        if (grid[row][col] == target) {
            break search;    // leaves both loops
        }
    }
}
```

Without it you need a flag variable and an extra condition on the outer loop, which is the kind of code that rots. Use it sparingly: more than one label in a method usually means the method wants splitting up.

Run It

```
java modules/03-control-flow/examples/OperatorsAndConditions.java
java modules/03-control-flow/examples/SwitchExpressions.java
java modules/03-control-flow/examples/WhyFallThroughWentAway.java
java modules/03-control-flow/examples/Loops.java

# Fails on purpose. Read which case it names.
java modules/03-control-flow/examples/SwitchMustBeExhaustive.java

./scripts/verify-examples.sh modules/03-control-flow
```

Common Mistakes

Getting the `&&` operands the wrong way round. `maybe.length() > 3 && maybe != null` throws before the null check ever runs.

Adding a `default` to a switch over an enum out of habit. It compiles today and hides every case you forget to add tomorrow.

Forgetting `break` in the old statement form. If you are writing new code, use the arrow form and the problem cannot occur.

Assuming compound assignment is just shorthand.

```
byte b = 10;
b += 300;      // compiles, prints 54
b = b + 300;   // does not compile
```

`+=` quietly inserts a cast. `10 + 300` is `310`, and truncating that to a byte gives `54`. The explicit form refuses instead, which is the safer of the two.

Using `==` on Strings built at runtime.

```
a1 == a2 : true      // both compile-time literals, pooled
a1 == a3 : false     // a3 was built by a StringBuilder
```

Same text, different answers, depending on when the string came into existence.

Writing a loop condition that relies on overflow.

```
for (int i = 1; i > 0; i *= 2) { }
```

This does terminate, but only because `i` wraps negative past `Integer.MAX_VALUE`, which is not the reason you wrote.

Key Takeaways

- Only a `boolean` goes in an `if`. There is no truthiness and no falsy table.
- `&&` and `||` short-circuit, which makes operand order a correctness concern.
- Switch expressions produce values, take several labels per branch, and never fall through.
- `yield` returns a value from a switch block; `return` would leave the method.
- Omit `default` on a closed set so that adding a case breaks the build instead of silently falling through.
- Use the enhanced `for` unless you need the index.
- `break label;` exits nested loops without a flag variable.

Homework

[homework/README.md](#)

Write a small grading tool with a switch expression, then convert a fall-through bug into a compile error. Reference solution in [solutions/03-control-flow/](#).